MCP

(Model Context Protocol)

Table of contents

📚 What is the Model Context Protocol (MCP)?

> The **Model Context Protocol (MCP)** is a standard that defines how **applications** (clients) and **servers** provide **contextual data, tools, and workflows** to **LLMs (Large Language Models)**.

Think of MCP like the **"USB for LLMs":** Just like USB standardizes how devices connect, MCP standardizes how **tools**, **resources**, and **prompts** connect to LLMs.

WHY DOES MCP MATTER?

`Without MCP`	`With MCP`
Each AI app has to invent its own way to send documents, actions, and prompts. It's messy, insecure, and hard to maintain.	Apps, databases, APIs, and models can speak a **common language**.
	Tools, resources, prompts are **discoverable**, **safe**, and **interoperable**.

An error occurred on this slide. Check the terminal for more information.

MCP Core Architecture Overview

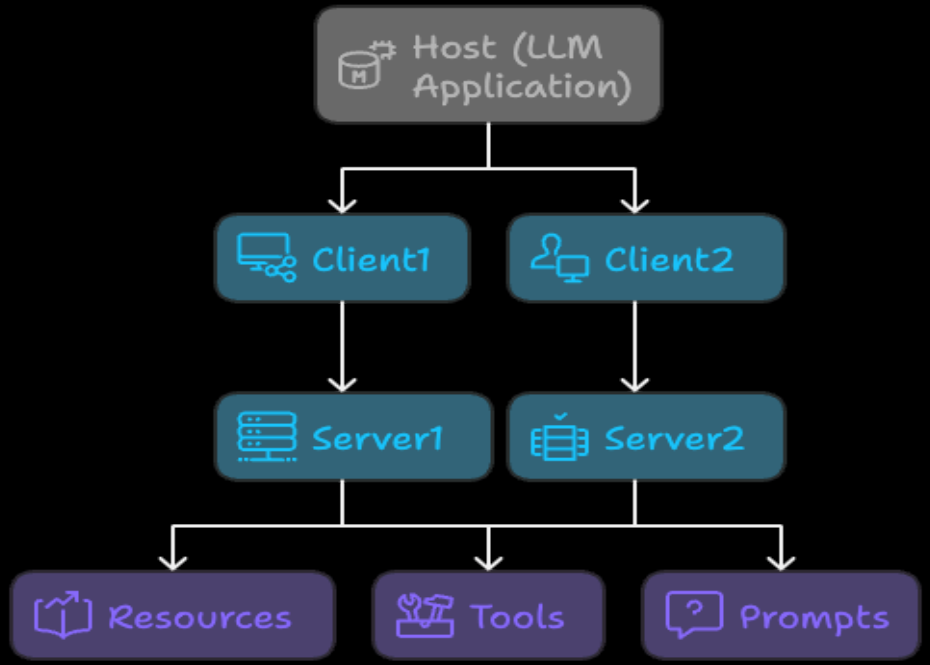
MCP is built around a **flexible, extensible client-server model** specifically optimized for LLM interactions.

🏗️ Architecture Overview

Component	Description
Host	The environment running the LLM (e.g., Claude Desktop).
Client	A bridge inside the host that manages 1:1 connections to servers.
Server	A backend that exposes **Resources**, **Tools**, and **Prompts** via the MCP protocol.

- The **Client** is embedded inside the Host (LLM app) and manages the communication.
- The **Server** is usually a Python app (or other language) that provides data and actions.

MCP Architecture Overview



MCP Core Components

The Model Context Protocol is structured around three main component layers:

- **Protocol Layer** (how we format and manage messages)
- **Transport Layer** (how we send/receive data)
- **Message Types** (the kinds of information exchanged)

Each layer is **independent and replaceable**, making MCP very modular and powerful for AI system designs.

An error occurred on this slide. Check the terminal for more information.

🚚 Transport Layer (Message Carrier)

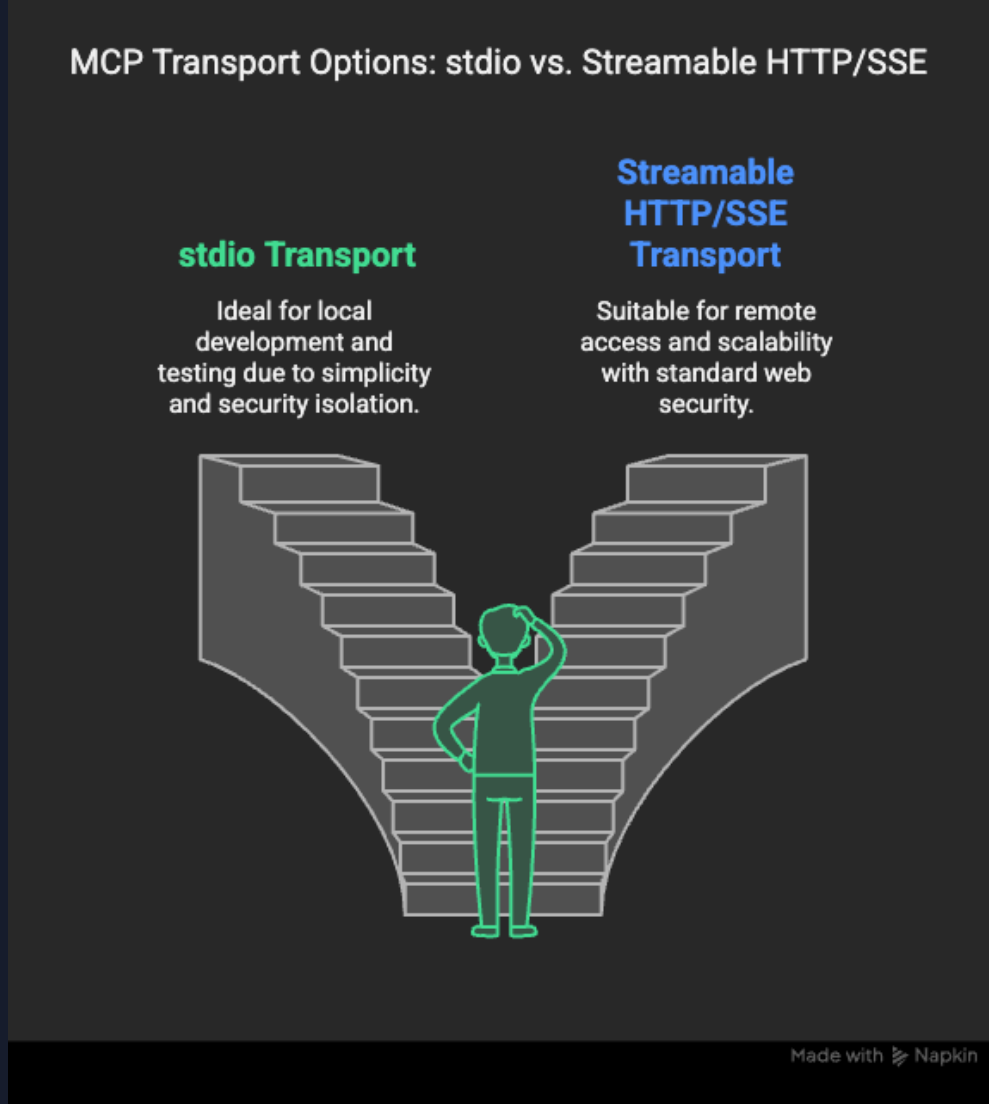
The **Transport Layer** defines **how the bytes actually travel** between Client and Server.

It **does not** care about the content – it only **moves JSON-RPC messages**.

✈️ Built-in Transport Types

Transport	Description	Use case
stdio	Standard input/output streams	Local scripts, command-line tools.
HTTP+SSE	Server-Sent Events + HTTP POST	Web apps, remote communication.

> Both use **JSON-RPC 2.0** for formatting.



JSON-RPC 2.0 Message Formats

Type	Example
Request	`{ "jsonrpc": "2.0", "id": 1, "method": "methodName", "params": { ... } }`
Response	`{ "jsonrpc": "2.0", "id": 1, "result": { ... } }`
Error	`{ "jsonrpc": "2.0", "id": 1, "error": { "code": -32600, "message": "Invalid Request" } }`
Notification	`{ "jsonrpc": "2.0", "method": "eventName", "params": { ... } }`

An error occurred on this slide. Check the terminal for more information.

MCP Connection Lifecycle

An error occurred on this slide. Check the terminal for more information.

Error Handling in Lifecycle

MCP defines standard error codes based on **JSON-RPC 2.0**:

Error Code	Meaning
`-32700`	Parse Error (Invalid JSON)
`-32600`	Invalid Request
`-32601`	Method Not Found
`-32602`	Invalid Params
`-32603`	Internal Error

Servers can define their own error codes > **-32000** for custom scenarios.

Python SDK Quickstart

🛠️ 1. Install the MCP Python SDK

```
pip install "mcp[cli]"
```

✅ This gives access to:

- MCP server libraries
- Development CLI tools (`mcp dev` , `mcp install` , etc.)

🧩 2. Create `server.py`

Here's the minimal working code:

```python
from mcp.server.fastmcp import FastMCP

# 1. Create the server
mcp = FastMCP("Demo Server")

# 2. Add a Resource
@mcp.resource("greeting://{name}")
def greet(name: str) -> str:
    """Return a personalized greeting."""
    return f"Hello, {name}! Welcome to MCP World."

# 3. Add a Tool
@mcp.tool()
def add_numbers(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b
```

✅ Highlights:

- `@mcp.resource()` exposes data (like a REST API GET).

🏃 3. Running the server locally

You can **develop and test** using the built-in CLI:

```
mcp dev server.py
```

✅ This starts a *local stdio server* and opens an **MCP Inspector UI** (in your browser) for easy testing!

If you want to run it directly:

```
python server.py
```

(But this will need a manual client to connect.)

Testing the server manually

Using `mcp` CLI:

```
mcp call-resource greeting://Alice
```

Should return:

```
{
  "text": "Hello, Alice! Welcome to MCP World."
}
```

Calling tool:

```
mcp call-tool add_numbers --args '{"a": 5, "b": 7}'
```

Should return:

```
{
  "text": "12"
}
```

📚 6. MCP Core Primitives: Resources, Prompts, Tools

Aspect	Resources	Prompts	Tools
Purpose	Provide static/dynamic context	Structure interaction	Perform actions
Control	Application-controlled	User-controlled	Model-controlled
Data	Text/Binary	Structured Messages	Action Outputs
Example	Fetch project docs	Guide code review	Create Jira Ticket

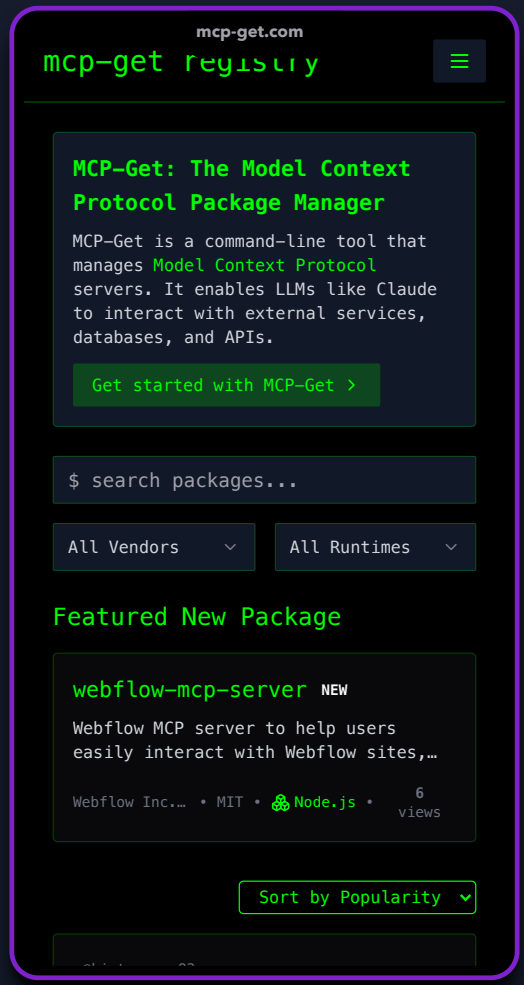
An error occurred on this slide. Check the terminal for more information.

An error occurred on this slide. Check the terminal for more information.

An error occurred on this slide. Check the terminal for more information.

An error occurred on this slide. Check the terminal for more information.


mcphub.io
Coming Soon!
Install mcp server easily - Client for Desktop
The mcp server, discovery
Find the right mcp server for you from 250+ servers collected
Describe your task in few words
eg: I need to look up flight information
AI Prompt
SEARCH
$ Popular searches
Post tweets and search for tweets by query
Manage Airtable bases
Retrieve information from the AWS Knowledge Base
Mcp Compass v1.0.0 | Powered by React & Next.js
2025 Mcp Club by Weight Wave


mcp-get.com
mcp-get registry
MCP-Get: The Model Context Protocol Package Manager
MCP-Get is a command-line tool that manages Model Context Protocol servers. It enables LLMs like Claude to interact with external services, databases, and APIs.
Get started with MCP-Get
$ search packages...
All Vendors
All Runtimes
Featured New Package
webflow-mcp-server NEW
Webflow MCP server to help users easily interact with Webflow sites,…
Webflow Inc.… • MIT • Node.js • 6 views
Sort by Popularity